SoulForms API

Documentación para integraciones externas (MCP, Power BI, scripts)

Versión del 2026-10-20

Base: `https://soulforms.com.co/api`

SoulForms – Documentación de la API para integraciones externas

Esta documentación describe el subconjunto de la API que un cliente externo (MCP de Claude, Power BI, scripts, otros servicios) puede consumir usando una **API Key** emitida desde el panel admin.

Todo el árbol **projects** → **folders** → **forms** se expone a la API key: lectura, creación y edición. Las únicas excepciones son borrar un formulario, publicarlo/despublicarlo y asignarlo, que siguen requiriendo JWT (ver sección 7). Toda la gestión del instancia (`/users` , `/groups` , `/api-keys` , `/consents` , `/tasks` , `/submissions` , `/reports`) sigue requiriendo JWT con sesión interactiva.

> **Importante**: no existe aislamiento por proyecto. Una API key cuyo owner sea `admin` ve y puede modificar **todo** el catálogo de SoulForms, no solo "sus" proyectos. Para acotar el alcance de una integración, el owner de la key debe ser un usuario con rol `coordinator` o `user` y permisos explícitos (`FORMS_CREATE` , `FORMS_EDIT`) — en ese caso solo podrá tocar los proyectos a los que esté asignado.

---

1. Autenticación

Esquema

Toda petición debe llevar el header:

```
x-api-key: sk_<64-hex-chars>
```

- Las API keys se crean en `/admin?section=api_keys` (solo admins).
- Al crearla, SoulForms muestra el valor completo **una sola vez**. Si se pierde, no se recupera: hay que generar otra.
- Se pueden revocar en el mismo panel. Una vez revocadas, cualquier petición posterior devuelve `401` .

Resolución del usuario

Internamente la key está ligada al usuario que la creó (`createdById`). Al validarla, el backend popula el contexto de la petición con el perfil de ese usuario, así los permisos y los campos "quién creó" quedan atribuidos a él. En particular `POST /forms` guarda `createdById = <owner de la key>` .

Códigos de error de auth

HTTP	Significado
`401 Unauthorized`	Key ausente, revocada, inexistente, o backend no reconoce el header.
`403 Forbidden`	Key válida, pero el usuario owner no tiene el permiso requerido (p. ej. `FORMS_CREATE` para `POST /forms`).

---

2. Base URL y convenciones

- **Producción**: `https://soulforms.com.co/api`
- **Content-Type** siempre: `application/json`
- Timestamps en ISO 8601 (`2026-10-16T12:34:56.789Z`)
- IDs: UUID v4 en todos los recursos (`_id` y `id` son el mismo valor; `id` es un alias que el `toJSON` del schema añade — usá `id` en el cliente).

---

3. Endpoints

3.1 Projects

Proyectos son el contenedor raíz. Un proyecto contiene carpetas, y una carpeta contiene formularios. **Todas** las operaciones de projects aceptan API key (no requieren JWT ni permisos adicionales — basta estar autenticado).

`GET /projects`

Lista los proyectos accesibles para el owner de la API key. Los usuarios con rol `admin` ven todos los proyectos activos; `coordinator` y `user` ven solo los proyectos a los que están asignados.

Respuesta `200 OK`:

```
[
  {
    "id": "1bbe99dd-2c44-4680-a1fd-46e89a051964",
    "_id": "1bbe99dd-2c44-4680-a1fd-46e89a051964",
    "name": "Inteligencia Digital",
    "color": "#3b82f6",
    "icon": "building",
    "isActive": true,
    "ownerId": 19,
    "createdAt": "2026-10-01T13:12:38.019Z",
    "updatedAt": "2026-10-01T13:12:38.019Z"
  }
]
```

`GET /projects/:id`

Devuelve un proyecto por id.

Respuesta `200 OK`: igual que un elemento de la lista. `404 Not Found` si no existe.

`POST /projects`

Crea un proyecto nuevo.

Body:

```
{
  "name": "Inteligencia Digital",   // string, requerido
  "color": "#3b82f6",               // string ≤20 chars, opcional
  "icon": "building"                // string ≤10 chars, opcional
}
```

Respuesta `201 Created`: el proyecto con su `id`, `ownerId` = owner de la key.

`PATCH /projects/:id`

Edita `name`, `color` o `icon` de un proyecto. No permite cambiar `ownerId` ni `isActive`.

Body (todos opcionales):

```
{ "name": "...", "color": "#...", "icon": "..." }
```

`DELETE /projects/:id`

Soft-delete del proyecto (`isActive = false`). Las carpetas y formularios anidados dejan de ser accesibles mientras el proyecto esté inactivo.

`POST /projects/:id/assign`

Asigna un usuario (no-admin) a un proyecto para que lo vea en su lista.

Body:

```
{ "userId": 42 }
```

`DELETE /projects/:id/assign/:userId`

Quita la asignación de un usuario al proyecto.

`GET /projects/:id/assignments`

Lista los usuarios asignados a un proyecto.

3.2 Folders

Carpetas lógicas dentro de un proyecto. Agrupan formularios por tema. **Todas** las operaciones de folders aceptan API key (no requieren permiso adicional).

`GET /folders?projectId=<uuid>&withForms=summary`

Lista las carpetas de un proyecto.

Query:

- `projectId` *(requerido)*: UUID del proyecto.
- `withForms=summary` *(opcional)*: incluye en cada carpeta el array `forms` (resumen liviano: `id`, `name`, `version`, `isPublic`, sin `schema`) y `formsCount`. Útil para evitar N+1 cuando se arma el árbol completo.

Respuesta `200 OK`:

```
[
  {
    "id": "52c5d8a1-4665-41f0-865e-1d1a2e4f4a8e",
    "_id": "52c5d8a1-4665-41f0-865e-1d1a2e4f4a8e",
    "name": "Consentimientos",
    "color": "#00c2a8",
    "icon": "📁",
    "isActive": true,
    "projectId": "1bbe99dd-2c44-4680-a1fd-46e89a051964",
    "createdAt": "2026-09-28T10:00:00.000Z",
    "updatedAt": "2026-10-01T13:12:38.019Z"
  }
]
```

`GET /folders/:id`

Devuelve una carpeta por id.

`POST /folders`

Crea una carpeta dentro de un proyecto.

Body:

```
{
  "name": "Nueva carpeta",     // string, requerido
  "projectId": "<uuid>",       // UUID, requerido
  "color": "#00c2a8",          // string ≤20 chars, opcional
  "icon": "📁"                  // string ≤10 chars, opcional
}
```

Respuesta `201 Created`: la carpeta creada con su `id`.

`PATCH /folders/:id`

Edita `name`, `color` o `icon`. **No permite cambiar** `projectId` — para mover una carpeta a otro proyecto hay que crearla de nuevo y borrar la anterior.

Body (todos opcionales):

```
{ "name": "...", "color": "#...", "icon": "..." }
```

`DELETE /folders/:id`

Soft-delete de la carpeta (`isActive = false`). Los formularios anidados dejan de ser accesibles mientras la carpeta esté inactiva.

3.3 Forms

Formularios dinámicos. Contienen un `schema` con la lista de widgets y reglas condicionales opcionales.

`GET /forms?folderId=<uuid>`

Lista los formularios de una carpeta.

Query:

- `folderId` *(requerido)*: UUID de la carpeta.

Respuesta `200 OK`: array de objetos `Form` (ver sección 4).

`GET /forms/:id`

Devuelve un formulario por id.

Respuesta `200 OK`: un objeto `Form` con el `schema` completo.

`404 Not Found` si no existe.

`POST /forms`

Crea un formulario nuevo. El owner de la API key debe tener permiso `FORMS_CREATE`.

Body:

```
{
  "name": "Consentimiento de vacunación",   // string, requerido
  "folderId": "<uuid>",                     // UUID, requerido
  "schema": {                               // objeto, opcional (default {widgets:[]})
    "widgets": [ /* ver sección 5 */ ],
    "rules":   [ /* ver sección 4.6, opcional */ ]
  },
  "emailTemplate": { /* ver sección 4.5, opcional */ }
}
```

Respuesta `201 Created`: el formulario creado, incluyendo el `id` autogenerado y `createdById = <owner de la key>`.

`403 Forbidden`: la key existe pero su user owner no tiene `FORMS_CREATE`.

`PATCH /forms/:id`

Edita un formulario existente. El owner de la API key debe tener permiso `FORMS_EDIT`. Cada edición incrementa `version` en 1 y queda registrada en auditoría (`AdminActionType.FORM_EDIT`).

Body (todos opcionales; sólo se envían los campos a modificar):

```
{
  "name":          "Consentimiento de vacunación v2",
  "schema":        { "widgets": [ ... ], "rules": [ ... ] },
  "emailTemplate": { ... }
}
```

Campos NO editables por este endpoint: `folderId`, `isPublic`, `sendConfirmationEmail`, `requiresEmailVerification`. Para publicar o despublicar un formulario y togglear el envío de confirmación hay que usar `PATCH /forms/:id/toggle-public`, que **sólo acepta JWT** y debe ejecutarse desde el panel admin.

Respuesta `200 OK`: el formulario completo tras la edición, con el `version` nuevo.

`403 Forbidden`: el owner no tiene `FORMS_EDIT`. `404 Not Found`: el formulario no existe o fue soft-deleted.

---

4. Shape de recursos

`Project`

```
{
  id: string;              // UUID (alias de _id)
  _id: string;             // UUID
  name: string;
  color: string;           // hex, ej "#3b82f6"
  icon: string;            // emoji o nombre de icono
  isActive: boolean;
  ownerId: number;         // id numérico del user owner
  createdAt: string;       // ISO 8601
  updatedAt: string;
}
```

`Folder`

```
{
  id: string;
  _id: string;
  name: string;
  color: string;
  icon: string;
  isActive: boolean;
  projectId: string;       // UUID del proyecto padre
  createdAt: string;
  updatedAt: string;
}
```

`Form`

```
{
  id: string;
  _id: string;
  name: string;
  folderId: string;        // UUID de la carpeta padre
  schema: {
    widgets: WidgetInstance[];
    rules?: unknown[];     // opcional, no documentado aquí
  };
  emailTemplate: object | null;   // config de email + PDF
  version: number;         // comienza en 1, se incrementa al editar
  isActive: boolean;
  createdById: number;     // id numérico del user que lo creó
  isPublic: boolean;       // si acepta submissions sin login
  sendConfirmationEmail: boolean;
  requiresEmailVerification: boolean;
  createdAt: string;
  updatedAt: string;
}
```

`WidgetInstance`

```
{
  id: string;              // UUID interno del widget dentro del form
  type: string;            // uno de los 26 tipos (ver sección 5)
  label: string;           // etiqueta visible junto al campo
  required: boolean;       // si es campo obligatorio
  config: Record<string, unknown>;  // configuración específica del tipo
}
```

4.5 `EmailTemplate`

Config completa del email + PDF que se envía al confirmar un submission. El backend lo persiste como JSON opaco; validación efectiva sucede al enviar.

```
{
  enabled: boolean;                // "toggle" general de envío
  subject: string;                 // puede contener placeholders ${campo}
  senderName?: string;
  replyTo?: string;

  toRecipients?:  EmailRecipient[]; // destinatarios principales
  ccRecipients?:  EmailRecipient[];
  bccRecipients?: EmailRecipient[];

  emailBody: string;               // HTML con placeholders ${campo}

  attachPDF:    boolean;           // si adjunta PDF
  pdfTemplate:  string;            // HTML del PDF con placeholders ${campo}
  pdfFilename:  string;            // puede llevar placeholders

  // Alternativas al pdfTemplate HTML (opcionales, se eligen por prioridad):
  pdfBase64?:   string;            // PDF base + mappings de coordenadas
  pdfMappings?: { placeholder: string; x: number; y: number;
                  page: number; fontSize?: number }[];
  excelBase64?: string;            // Excel base + mappings; se convierte a PDF
  excelMappings?: { coord: string; placeholder: string }[];
  excelFilename?: string;
  excelLogoBase64?: string;
}
```

`EmailRecipient`

Dos tipos — **y sólo dos**:

```
type EmailRecipient =
  | { type: 'static'; email: string }
  | { type: 'group';  group: Group; groupLabel?: string };

type Group = 'admin' | 'coordinator' | 'user' | 'current_user' | 'me';
```

Resolución del `group` (backend/src/email/email.service.ts:272-301):

`group`	A quién le llega
`admin`	Todos los usuarios con rol `admin`, activos.
`coordinator`	Todos los usuarios con rol `coordinator`, activos.
`user`	Todos los usuarios con rol `user`, activos.
`me`	El usuario que envía el submission (si aplica).
`current_user`	El primer valor del submission que matchea `^[^\s@]+@[^\s@]+\.[^\s@]+$`; si no encuentra ninguno, cae al usuario que envía.

> **Enviar al email que el paciente escribió en el formulario**: usar `{ type: 'group', group: 'current_user' }`. Si el formulario tiene varios campos email, se tomará el **primero** encontrado (orden de widgets). No existe actualmente un tipo `field` ni `catalog` para apuntar a un `widgetId` específico.

Placeholders `${campo}`

Dentro de `subject`, `emailBody`, `pdfTemplate`, `pdfFilename` y los `mappings` de PDF/Excel, cada `${campo}` se reemplaza por el valor del widget cuyo `label` **normalizado** coincide. Normalización (pdf-interpolator.ts:32-34):

```
label.toLowerCase().replace(/\s+/g, '').replace(/[^a-z0-9]/gi, '')
```

Ejemplos:

Label del widget	Placeholder a usar
`Nombre completo`	`${nombrecompleto}`
`Teléfono del paciente`	`${telefonodelpaciente}` *(la `é` cae por el filtro)*
`Documento de Identidad`	`${documentodeidentidad}`
`Fecha`	`${fecha}`

Las firmas y fotos se almacenan en el submission como `gridfs:<fileId>` y el interpolador las embebe automáticamente como `<img src="data:...">` en el HTML del PDF — en el template simplemente poné `${firmadelpaciente}`.

4.6 `FormRule`

Reglas condicionales que muestran/ocultan widgets según los valores ingresados. Viven dentro de `schema.rules` y son opcionales. El backend NO valida su shape; la validación efectiva ocurre en el renderer del frontend (src/types/widget.types.ts:3-24).

```
type FormRule = {
  id: string;                       // único dentro del form (UUID o slug)
  name: string;                     // descriptivo, visible en el panel admin
  matchType: 'all' | 'any';         // AND / OR entre conditions
  conditions: RuleCondition[];
  action: 'show' | 'hide';
  targetWidgetIds: string[];        // widgets afectados por la acción
};

type RuleCondition = {
  widgetId: string;                 // apunta a WidgetInstance.id
  operator: RuleOperator;
  value: string;                    // vacío para operadores sin valor
};

type RuleOperator =
  | 'equals' | 'not_equals' | 'contains'
  | 'not_empty' | 'is_empty' | 'has_value';
```

> Importante: los `widgetId` dentro de `conditions[].widgetId` y `targetWidgetIds[]` deben existir en `schema.widgets[].id`. Si no, la regla simplemente no hace nada en el renderer.

---

5. Catálogo de widgets

Los 26 tipos disponibles con su `defaultConfig`. Al crear un widget se puede omitir `config` (se aplicará el default) o pasar un objeto parcial que se mergea sobre el default.

type	Label visible	Descripción
`text`	Texto	Input de una línea con restricciones de longitud y clase de caracteres.
`textarea`	Párrafo	Multi-línea con control de longitud y filas.
`number`	Número	Numérico con cotas min/max/step y decimales opcionales.
`email`	Email	Input de email con validación.
`phone`	Teléfono	Con selector de país (LATAM+ES+US).
`date`	Fecha	Selector de fecha con cotas opcionales.
`time`	Hora	HH:MM, opcional pre-llenado con hora actual.
`select`	Desplegable	Dropdown single/multi, opcional expandido.
`radio`	Radio	Radio buttons, opciones pueden tener imagen.
`checkbox`	Casillas	Grupo de checkboxes.
`signature`	Firma	Pad de firma manuscrita.
`photo`	Foto	Captura/subida con tope de tamaño.
`file`	Archivo	Subida genérica con límite.
`video`	Video	Captura/subida con duración y tamaño.
`image`	Imagen	Imagen estática en el form.
`header`	Encabezado	Encabezado de sección.
`html_block`	Bloque HTML	HTML enriquecido (branding, legales).
`subform`	Subformulario	Grupo repetible anidado.
`search`	Búsqueda	Typeahead contra submissions/Sheets/Excel/SQL.
`calc`	Cálculo	Campo read-only sum/avg/etc sobre otros widgets.
`price`	Precio	Entrada monetaria con formato.
`barcode`	Código barras	Scanner de códigos de barras.
`id_scanner`	Cédula (OCR)	Escanea cédulas CO (PDF417/MRZ).
`id_document`	Documento ID	Entrada de documento por país.
`quiz_question`	Pregunta evaluada	Con opciones puntuadas.
`quiz_score`	Nota final	Agrega puntos de quiz_question.

`defaultConfig` por tipo

Los defaults completos están en el MCP (`tools/soulforms-mcp/src/widget-catalog.ts`) y también se obtienen vía la tool `list_widget_types`. Resumen de los más usados:

```
// text
{ "placeholder":"", "defaultValue":"", "maxLength":100,
  "allowNumbers":false, "allowSpecialChars":false, "allowLineBreaks":false }

// textarea
{ "placeholder":"", "maxLength":500, "rows":4 }

// number
{ "placeholder":"", "min":0, "max":999999, "step":1, "allowDecimals":false }

// email
{ "placeholder":"correo@ejemplo.com" }

// phone
{ "placeholder":"300 123 4567", "prefix":"+57", "maxLength":10,
  "enableCountrySelector":false, "defaultCountry":"CO" }

// date
{ "min":"", "max":"" }

// time
{ "nowByDefault":false, "rememberInput":false }

// select
{ "placeholder":"Selecciona una opción",
  "options":["Opción 1","Opción 2","Opción 3"],
  "defaultValue":"", "sortAlphabetically":false,
  "multiple":false, "expand":false, "rememberEntry":false }

// radio
{ "options":[{"id":"opt_1","label":"Opción 1"},{"id":"opt_2","label":"Opción 2"}],
  "useImageLabels":false, "defaultValue":"",
  "verticalAlign":true, "rememberInput":false }

// checkbox
{ "options":["Opción 1","Opción 2","Opción 3"] }

// signature
{ "penColor":"#000000", "penWidth":2 }

// photo
{ "allowCamera":true, "allowUpload":true, "maxSizeMB":5 }

// file
{ "fileType":"documents", "maxSizeMB":25 }

// header
{ "text":"Sección", "size":"medium", "showDivider":true }

// subform
{ "addButtonLabel":"Agregar", "fields":[], "minEntries":0, "maxEntries":0 }
```

6. Ejemplos end-to-end

6.1 Listar proyectos

```
curl -H "x-api-key: sk_abc123..." \
     https://soulforms.com.co/api/projects
```

6.2 Crear carpeta

```
curl -X POST -H "x-api-key: sk_abc123..." \
     -H "Content-Type: application/json" \
     -d '{
       "name": "Formularios clínicos",
       "projectId": "1bbe99dd-2c44-4680-a1fd-46e89a051964"
     }' \
     https://soulforms.com.co/api/folders
```

6.3 Crear formulario con 3 widgets

```
curl -X POST -H "x-api-key: sk_abc123..." \
     -H "Content-Type: application/json" \
     -d '{
       "name": "Consentimiento de vacunación",
       "folderId": "52c5d8a1-4665-41f0-865e-1d1a2e4f4a8e",
       "schema": {
         "widgets": [
           {
             "id": "w_fecha",
             "type": "date",
             "label": "Fecha",
             "required": true,
             "config": { "min": "", "max": "" }
           },
           {
             "id": "w_nombre",
             "type": "text",
             "label": "Nombre del paciente",
             "required": true,
             "config": { "placeholder": "", "defaultValue": "",
                         "maxLength": 100, "allowNumbers": false,
                         "allowSpecialChars": false, "allowLineBreaks": false }
           },
           {
             "id": "w_firma",
             "type": "signature",
             "label": "Firma del paciente",
             "required": true,
             "config": { "penColor": "#000000", "penWidth": 2 }
           }
         ],
         "rules": []
       }
     }' \
     https://soulforms.com.co/api/forms
```

6.4 Verificar formulario creado

```
curl -H "x-api-key: sk_abc123..." \
     https://soulforms.com.co/api/forms/<id-devuelto>
```

6.5 Editar formulario (schema + email)

```
curl -X PATCH -H "x-api-key: sk_abc123..." \
     -H "Content-Type: application/json" \
     -d '{
       "name": "Consentimiento de vacunación v2",
       "schema": {
         "widgets": [ /* ... misma lista o modificada ... */ ],
         "rules": [
           {
             "id": "r_1",
             "name": "Ocultar motivo si acepta",
             "matchType": "all",
             "conditions": [
               { "widgetId": "w_acepta", "operator": "equals", "value": "Sí" }
             ],
             "action": "hide",
             "targetWidgetIds": ["w_motivo_rechazo"]
           }
         ]
       },
       "emailTemplate": {
         "enabled": true,
         "subject": "Consentimiento firmado — ${nombredelpaciente}",
         "senderName": "Sara Pacientes",
         "toRecipients": [
           { "type": "group", "group": "current_user" },
           { "type": "group", "group": "admin" }
         ],
         "emailBody": "<p>Hola ${nombredelpaciente}, adjuntamos tu consentimiento.</p>",
         "attachPDF": true,
         "pdfTemplate": "<h1>Consentimiento</h1><p>Nombre: ${nombredelpaciente}</p><p>Fecha: ${
         "pdfFilename": "Consentimiento_${nombredelpaciente}.pdf"
       }
     }' \
     https://soulforms.com.co/api/forms/<id-devuelto>
```

Requiere `FORMS_EDIT` en el owner de la key. Incrementa `version` y queda registrado en el log de auditoría.

---

7. Lo que NO se puede hacer con API key

Por diseño, estas operaciones requieren JWT (sesión interactiva de admin) y devuelven `401` si se intentan con `x-api-key`:

Sobre formularios (3 excepciones puntuales):

- `DELETE /forms/:id` — eliminar formulario. Comentario del código: "*una key fugada no debería poder eliminar*" (forms.controller.ts:97).

- `PATCH /forms/:id/toggle-public` — publicar/despublicar y togglear `sendConfirmationEmail` / `requiresEmailVerification`.
- `POST /forms/:id/assign` — asignar formulario a usuarios o grupos.

Toda la gestión de instancia:

- `/users` — crear, editar, borrar usuarios, resetear 2FA.
- `/groups` — crear grupos, agregar/quitar miembros, asignar formularios a grupos.
- `/api-keys` — emitir, listar, revocar keys (una key no puede crear más keys).
- `/consents` — definiciones de consentimientos (seeder admin).
- `/tasks` — tareas programadas, envíos one-shot, share links, reenvíos.
- `/submissions` — editar o anular registros ya enviados.
- `/reports` — generar y descargar reportes/exports.
- `/admin-audit` — log de auditoría administrativa.

Para automatizar algo de esta lista (ej. publicar un formulario desde el MCP), hay que ampliar el guard del endpoint y auditar explícitamente el cambio. Pedir al equipo de backend.

> **Blast radius de una API key filtrada**: aunque `DELETE /forms/:id` esté bloqueado, la key sí puede soft-deletear proyectos y carpetas enteras — esto deja a los formularios anidados inaccesibles aunque sigan existiendo en la DB. Recuperación: un admin reactiva el proyecto/carpeta desde el panel. Mitigación preventiva: **revocar la key** desde `/admin?section=api_keys` apenas se sospecha fuga, y rotar.

---

8. Convenciones para generar un formulario correcto

Reglas a seguir cuando se construye un `widgets[]` desde fuera (ej. clonando desde otra plataforma):

1. `id`: debe ser único dentro del formulario; usar UUID o un slug descriptivo (`w_<campo>`). Si se omite al crear, el backend lo asigna.
2. `type`: debe ser uno de los 26 strings exactos del catálogo. Un valor desconocido rompe el renderer.
3. `label`: obligatorio y visible al usuario final.
4. `required`: boolean. Default `false`.
5. `config`: puede omitirse completo o pasar solo las claves que uno quiere sobrescribir. El resto se completa con `defaultConfig`.
6. **Orden**: los widgets se renderizan en el orden del array. No existe un `position` separado.
7. **Opciones de** `select`: array de strings.
8. **Opciones de** `radio`: array de objetos `{id, label, imageUrl?}`. Si se pasa un array de strings, el backend lo normaliza al momento de servir.
9. **Opciones de** `checkbox`: array de strings.

10. `rules`: opcional (sección 4.6). Cada `conditions[].widgetId` y cada entrada de `targetWidgetIds[]` debe apuntar a un `widget.id` existente; si no, la regla no hace nada. Si no se usan reglas, mandar `rules: []`.
11. `emailTemplate`: opcional (sección 4.5). Si se pasa con `enabled: true` y `toRecipients.length > 0`, el backend enviará el email al confirmar cada submission. Dentro de `subject`, `emailBody`, `pdfTemplate` y `pdfFilename` usar placeholders `${campo}` donde `campo = normalize(label)` (lowercase, sin espacios, sin tildes ni símbolos — ver 4.5). Para publicar el formulario (que acepte submissions sin login) hay que ir al panel admin: la API key no puede cambiar `isPublic` ni `sendConfirmationEmail`.

---

9. Troubleshooting

Síntoma	Causa probable	Fix
`401 Unauthorized` global	Key revocada o no enviada	Verificar que `x-api-key` esté en el header y la key esté activa en el panel admin.
`403 Forbidden` en `POST /forms`	Owner sin `FORMS_CREATE`	Un admin tiene que agregarle el permiso al user owner de la key.
`400 Bad Request` en `POST`	DTO inválido (campos faltantes o tipos incorrectos)	Revisar que `name` esté presente, `folderId` / `projectId` sean UUID válidos.
`404 Not Found` en `GET /:id`	El recurso fue borrado (soft/hard) o el id es incorrecto	Verificar que el id exista con el `GET` de lista.
`500 Internal Server Error`	Error inesperado del backend	Pegar el error completo y el curl que lo reprodujo al equipo.

---

10. Matriz completa de endpoints

10.1 Lo que SÍ puede hacer una API key

Método	Path	Permiso requerido	Notas
Projects			
`GET`	`/projects`	—	Admin ve todos; coordinator/user solo los asignados.
`GET`	`/projects/:id`	—	Detalle.
`POST`	`/projects`	—	Crear proyecto.
`PATCH`	`/projects/:id`	—	Editar `name`, `color`, `icon`.
`DELETE`	`/projects/:id`	—	Soft-delete.
`POST`	`/projects/:id/assign`	—	Asignar usuario al proyecto.
`DELETE`	`/projects/:id/assign/:userId`	—	Quitar asignación.
`GET`	`/projects/:id/assignments`	—	Lista de asignados.
Folders			
`GET`	`/folders?projectId=<uuid>`	—	Lista carpetas; `?withForms=summary` incluye forms resumidos.
`GET`	`/folders/:id`	—	Detalle.
`POST`	`/folders`	—	Crear carpeta.
`PATCH`	`/folders/:id`	—	Editar `name`, `color`, `icon`.
`DELETE`	`/folders/:id`	—	Soft-delete.
Forms			
`GET`	`/forms?folderId=<uuid>`	—	Lista formularios de una carpeta.
`GET`	`/forms/:id`	—	Detalle con `schema` completo.
`POST`	`/forms`	`FORMS_CREATE`	Crear formulario.
`PATCH`	`/forms/:id`	`FORMS_EDIT`	Editar `name`, `schema` (widgets + rules), `emailTemplate`. Incrementa `version`.

10.2 Lo que NO puede hacer una API key (solo JWT)

Método	Path	Por qué
`DELETE`	`/forms/:id`	Mitigación de key fugada: no debería poder borrar formularios.
`PATCH`	`/forms/:id/toggle-public`	Publicar/despublicar y togglear `sendConfirmationEmail` / `requiresEmailVerification`.
`POST`	`/forms/:id/assign`	Asignar formulario a usuarios/grupos.
`*`	`/users/*`	Gestión de usuarios y 2FA.
`*`	`/groups/*`	Gestión de grupos.
`*`	`/api-keys/*`	Una key no puede emitir ni revocar otras keys.
`*`	`/consents/*`	Seeder y gestión de definiciones de consentimiento.
`*`	`/tasks/*`	Tareas programadas, envíos, share links.
`*`	`/submissions/*`	Editar/anular registros enviados.
`*`	`/reports/*`	Generación y descarga de reportes/exports.
`*`	`/admin-audit/*`	Log de auditoría.

10.3 Matriz CRUD rápida

Recurso	Create	Read	Update	Delete	Publicar	Asignar
Projects	✅	✅	✅	✅	n/a	✅
Folders	✅	✅	✅	✅	n/a	n/a
Forms	✅	✅	✅	❌	❌	❌
Users	❌	❌	❌	❌	n/a	n/a
Groups	❌	❌	❌	❌	n/a	n/a
API Keys	❌	❌	n/a	❌	n/a	n/a

Fin de la documentación.